

Redução Transitiva e Conservação de Alcançabilidade de Vértices em Grafos

Davi Cândido de Almeida [✉] [Pontifícia Universidade Católica de Minas Gerais | davicandidopucminas@gmail.com]

Kaiky França dos Reis [✉] [Pontifícia Universidade Católica de Minas Gerais | kaikyfrs.comp@gmail.com]

Luís Fernando Rodrigues Braga [✉] [Pontifícia Universidade Católica de Minas Gerais | luisbraga87@hotmail.com]

Priscila Goulart Carvalho [✉] [Pontifícia Universidade Católica de Minas Gerais | pgc080595@gmail.com]

Suzane Lemos de Lima [✉] [Pontifícia Universidade Católica de Minas Gerais | suzy.lemosdelima@hotmail.com]

Samuel Horta de Faria [✉] [Pontifícia Universidade Católica de Minas Gerais | samuel.fariacc@gmail.com]

Victor Monteiro Martinelli Grataroli [✉] [Pontifícia Universidade Católica de Minas Gerais | victormartinellig@gmail.com]

[✉] Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500 - Coração Eucarístico, Belo Horizonte - MG, 30535-901, Brazil.

Received: 22 Maio 2026 • Accepted: DD Month YYYY • Published: 28 Junho 2026

Resumo. A redução transitiva é fundamental na teoria dos grafos e computabilidade (TGC) por simplificar a estrutura de grafos por meio da eliminação de arestas redundantes enquanto é preservada a alcançabilidade entre todos os pares de vértices. Neste trabalho, será apresentada e desenvolvida uma análise sobre a redução transitiva em grafos baseada em C++. Será observado também como as escolhas de representação e tipo de grafo impactam na solução final do problema, também será comparado o efeito do uso de matrizes de adjacência no custo computacional em comparação com abordagens já estabelecidas em artigos anteriores usados como base teórica. Também serão abordados os impactos e as implicações geradas para uma variedade de topologias de grafos.

Keywords: Grafo, TGC, Transitividade, Alcançabilidade, Matriz de Adjacência, DAG, Warshall, Grafo Direcionado, Grafo Não-Direcionado

1 Introdução

A modelagem de sistemas complexos por meio de estruturas de grafos $G = (V, E)$ é uma prática que permite a resolução de problemas que envolvem redes de transporte, fluxos de informação e dependências estruturais de vértices. Na prática, o custo computacional associado a algoritmos de caminho e análise de conectividade está diretamente associado à densidade de arestas do grafo. Quanto maior o número de vértices, maior é a presença de arestas redundantes no grafo, o que por sua vez acaba aumentando desnecessariamente o tempo de processamento e consumo de memória de busca e classificação dos vértices.

Nesse contexto, o processo de redução transitiva em um grafo é uma etapa essencial para a redução de complexidade e de custos computacionais durante o processo de caminho em um grafo genérico Jerônimo *et al.* [2025]. O processo em si de caminho e classificação é trabalhado em torno de grafos direcionados, já que o objetivo é a eliminação de arestas vistas como redundantes, mas que ainda assim preservam a alcançabilidade de um par de vértices para os demais. Essa operação fundamental e suas implicações estruturais foram formalizadas originalmente no trabalho de Aho *et al.* [1972].

Quando fazemos uma análise mais pontual em grafos direcionados, percebemos que embora a conectividade geral seja preservada, a remoção dessas arestas podem introduzir trade-offs, como o aumento do comprimento de caminhos mínimos entre determinados pares de vértices. Além disso, seguindo outros artigos, a identificação e remoção dessas arestas re-

dundantes implicam em um custo computacional que depende da representação e topologia do grafo em análise, mesmo que seja preservada a alcançabilidade de um par de vértices para os demais.

Diante desse cenário, o presente trabalho apresenta o desenvolvimento e o comportamento de duas estratégias para a redução transitiva. A primeira consiste na abordagem puramente algébrica de Aho *et al.* [1972], que se apoia na manipulação de matrizes para a extração do fecho transitivo. A segunda, desenvolvida especificamente para atender à delimitação deste estudo, adota a navegação estrutural por meio de caminho no grafo. Sob essa ótica comparativa, analisa-se o custo computacional de ambas as implementações diante da variação de densidade das instâncias testadas.

2 Metodologia Proposta

As estratégias desenvolvidas neste trabalho para solucionar o problema de redução transitiva compartilham de uma base conceitual comum, de que ambas operam por meio de algoritmos gulosos e exigem um tratamento prévio de ciclos para garantir a solução adequada. Para isso, ambas as abordagens iniciam executando o algoritmo de Tarjan, cuja complexidade computacional é $O(|V| + |E|)$, com o objetivo de identificar e comprimir os componentes fortemente conexos. Essa etapa intermediária condensa o grafo original em um Grafo Direcionado Acíclico (DAG) de componentes. Essa imposição de aciclicidade é o alicerce que viabiliza o método guloso: além de garantir matematicamente que a redução

transitiva seja estritamente determinística e única, permite que as decisões de remoção de arestas sejam tomadas de forma definitiva, eliminando o custo do *backtracking*.

2.1 Abordagem por Operações Algébricas de Matrizes

A estratégia de Matrizes é baseada na manipulação algébrica estrutural do grafo empregando uma representação por Matriz de Adjacência. O funcionamento padrão do algoritmo realiza algumas fases:

- **Identificar SCC e criar supervértices:** Inicialmente o algoritmo de Tarjan é executado em tempo $O(|V|+|E|)$, para identificar as Componentes Fortemente Conexas de um grafo (SCC - *Strongly Connected Component*). Cada componente então é condensada em um supervértice, transformando a estrutura original, em um Grafo Direcionado Acíclico (DAG).
- **Cálculo do Fecho Transitivo:** Estabelecido o Grafo Direcionado Acíclico (DAG) de supervértices, aplica-se o método de Warshall sobre a matriz representativa desse DAG para mapear o fecho transitivo completo entre todos os pares de componentes. O custo desta operação é dado por três laços aninhados que percorrem o número total de vértices presentes ($|V|$), gerando a complexidade de $O(|V|^3)$.
- **Poda Gulosa por Análise Matricial:** A etapa de poda avalia sistematicamente as conexões do grafo. O critério guloso estabelece que, para uma aresta direta (u, v) , caso exista um vértice intermediário w ($w \neq u$ e $w \neq v$) tal que haja uma aresta de u para w e a matriz de fecho transitivo indique que w alcança v , a aresta original (u, v) é classificada como redundante. Satisfeita essa condição, a conexão é imediatamente suprimida. A estrutura dessa validação exige três laços encadeados sobre o conjunto de vértices, resultando em uma complexidade assintótica de $O(|V|^3)$.

2.2 Abordagem por Caminhamento no Grafo

A estratégia utiliza estruturas de lista de Adjacência e caminhamento no grafo. Para garantir a qualidade da solução e evitar que caminhos circulares escondam a real alcançabilidade entre os vértices, essa abordagem exige o tratamento de ciclos:

- **Condensação em DAG via Algoritmo de Tarjan:** O algoritmo executa previamente Tarjan, para encontrar os componentes fortemente conexos com complexidade $O(|V| + |E|)$. Os componentes então são identificados e contraídos em "supervértices", gerando um Grafo Direcionado Acíclico intermediário.
- **Poda Gulosa por Busca em Profundidade (DFS):** O algoritmo percorre cada aresta direcionada do DAG gerado pelos componentes. Para cada aresta avaliada, executa-se um teste de remoção provisória. No caso, a aresta analisada é removida temporariamente da estrutura de listas DAG. Executa-se um caminhamento de Busca em Profundidade (DFS) a partir do nó de origem,

cujo custo é $O(|V_{scc}| + |E_{scc}|)$. Caso o caminhamento indique que o vértice vizinho continua a ser alcançável por outros caminhos alternativos, a decisão gulosa de exclusão é mantida. Caso contrário, a aresta é reinserida.

- **Reconstrução (Expansão Cíclica):** Ao final, o grafo é restabelecido expandindo os componentes fortemente conexos em ciclos simples sequenciais. Matematicamente, um ciclo simples de N vértices contém exatamente N arestas, o que representa o número mínimo absoluto necessário para preservar a atingibilidade mútua entre todos os nós do componente. Por fim, as arestas de trânsito remanescentes são religadas a partir do vértice com o menor índice de cada agrupamento.

3 Comparação Direta de Desempenho e Estruturas

A eficiência prática das abordagens desenvolvidas não depende isoladamente da complexidade assintótica, mas sim de como o padrão de acesso à memória de cada estrutura reage à densidade e topologia do grafo. Nesse caso, quando comparamos ambas as abordagens seja por Matriz de Adjacência e por Lista de Adjacência, pode ser percebido de forma mais clara como ambas realizam a redução transitiva.

No caso, podemos perceber através da imagem um exemplo de grafo com componentes fortemente conexos que será trabalhado como exemplo pelos algoritmos

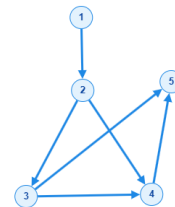


Figure 1. Exemplo de grafo com componentes fortemente conexos.

3.1 Comportamento em Grafos Esparsos

Grafos esparsos são caracterizados por possuírem um número de arestas muito próximo ou proporcional ao número de vértices ($|E| \approx |V|$).

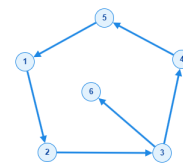


Figure 2. Exemplo de grafo esparso.

- A abordagem por Lista de Adjacência: Apresenta desempenho significativamente superior neste cenário. Como a estrutura armazena estritamente as conexões reais, a Busca em Profundidade (DFS) gulosa consome

apenas o tempo necessário para percorrer os vizinhos existentes, operando em $O(|V_{scc}| + |E_{scc}|)$ por chamada. O uso de memória também é otimizado, evitando o desperdício com posições nulas.

- Já Abordagem por Matriz de Adjacência: Torna-se ineficiente em estruturas esparsas. Mesmo que o grafo possua poucas arestas, os três laços aninhados do algoritmo de Warshall e da poda matricial varrem obrigatoriamente todas as coordenadas da matriz ($|V_{scc}|^3$), inspecionando exaustivamente células preenchidas com zeros, gerando um custo computacional fixo e desnecessário.

3.2 Comportamento em Grafos Densos ou Completos

Quando analisamos grafos densos, onde o número de arestas se aproxima do limite quadrático em relação ao número de vértices ($|E| \approx |V|^2$), o comportamento dos algoritmos se altera significativamente, impactando a lógica de poda das arestas redundantes.

- A **abordagem por Matriz de Adjacência** apresenta um comportamento assintótico estável de $O(|V_{scc}|^3)$ devido aos laços aninhados do algoritmo de Warshall e da poda matricial. Em cenários densos, embora o custo computacional global permaneça elevado, essa estrutura beneficia-se do acesso direto à memória em tempo constante $O(1)$. Isso torna o limite cúbico previsível e consideravelmente mais eficiente do que o processamento baseado em listas.
- A **abordagem por Lista de Adjacência**, por outro lado, sofre uma degradação severa, representando o pior cenário para a resolução. O algoritmo inspeciona cada aresta e executa uma Busca em Profundidade (DFS), resultando em um custo de $O(|E_{scc}| \times (|V_{scc}| + |E_{scc}|))$. Como neste caso o número de arestas alcança o limite quadrático ($|E_{scc}| \approx |V_{scc}|^2$), a substituição matemática demonstra que a complexidade se eleva para $O(|V_{scc}|^2 \times (|V_{scc}| + |V_{scc}|^2))$, o que assintoticamente domina e consolida a complexidade de ordem $O(|V|^4)$.

4 Análise de Comportamento em Grafos Não-Direcionados

A aplicação do conceito de redução transitiva em grafos não-direcionados perde a propriedade de unicidade. Isso ocorre porque, nesse cenário, as relações de adjacência são bidirecionais: se existe um caminho de A até B, automaticamente existe um caminho de B até A. Consequentemente, a alcançabilidade torna-se reflexiva, simétrica e naturalmente transitiva ao longo de qualquer caminho válido.

Em termos práticos, essa simetria colapsa o critério de escolha gulosa utilizada pelos algoritmos, já que passa a gerar soluções ambíguas que dependem da ordem de varredura das arestas e de qual é a topologia do grafo Sidford [2018]. Por exemplo, ao avaliar um ciclo simples composto por três vértices mutuamente conectados (A, B e C), se a aresta {A, C}

for visitada primeiro, o algoritmo a classificará como "redundante" devido à existência de um caminho alternativo linear passando por {A, B} e {B, C}. No entanto, as arestas {A, B} ou {B, C} possuiriam exatamente a mesma justificativa matemática para remoção caso fossem analisadas no início.

Essa ausência de orientação temporal ou causal impede o algoritmo de estabelecer uma hierarquia de dependência entre os vértices, destruindo a unicidade da redução transitiva inerente às estruturas acíclicas direcionadas (DAGs).

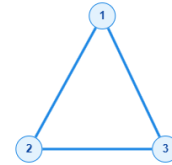


Figure 3. Grafo não direcionado

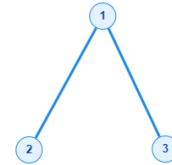


Figure 4. Aplicação da redução transitiva em um grafo não direcionado

Na teoria em grafos, remover o maior número possível de arestas de um grafo não direcionado mantendo a conectividade global entre os nós não é chamado de redução transitiva, mas sim de busca por Árvore Geradora (Spanning Tree). Caso o grafo possua pesos, o problema passa a ser o de encontrar a Árvore Geradora Mínima (MST), onde a solução é dada por algoritmos como Kruskal ou Prim, cujas propriedades matemáticas e estrutura diferem da abordagem gulosa de redução transitiva em grafos direcionados como abordados neste trabalho.

5 Resultados Obtidos

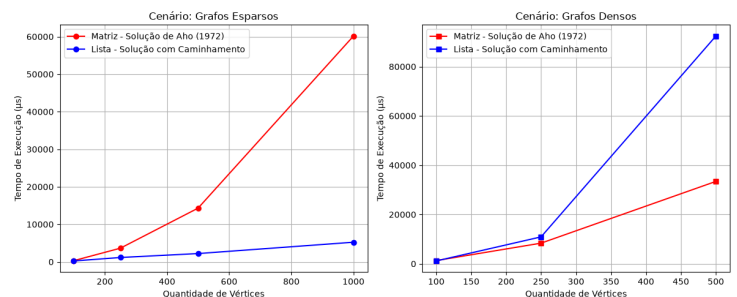


Figure 5. Gráfico comparativo entre grafos Esparsos e Densos

Os testes computacionais realizados com ambas as implementações permitiram mapear de forma clara os limites operacionais e a eficiência geral de cada estrutura de dados frente à densidade dos grafos analisados pelos algoritmos.

A abordagem fundamentada em Lista de Adjacência demonstrou alta eficiência em cenários de grafos esparsos.

Table 1. Comparativo de tempo de execução (μs) entre as abordagens testadas.

Topologia	Vértices	Arestas	Tempo Matriz	Tempo Lista
Esparsa	100	250	322	242
	250	600	3.644	1.162
	500	1.200	14.310	2.211
	1000	2.500	60.147	5.232
Densa	100	8.000	1.306	1.153
	250	55.000	8.376	10.921
	500	220.000	33.423	92.391

Como a Busca em Profundidade (DFS) gulosa inspeciona apenas as conexões reais existentes, o tempo de execução permaneceu reduzido quando o número de arestas era consideravelmente menor que o quadrado do número de vértices ($|E| \ll |V|^2$). Contudo, sob condições de alta densidade, essa estrutura sofreu uma degradação severa de desempenho, aproximando-se do limite assintótico de pior caso de $O(|V|^4)$ devido ao custo do percurso linear em listas extensas para cada aresta avaliada.

Em contrapartida, a representação por Matriz de Adjacência desenvolvida para o trabalho, exibiu um comportamento inverso, caracterizado por uma maior estabilidade assintótica em cenários com grafos densos. Embora o algoritmo de Warshall imponha um custo fixo inicial na ordem de $O(|V_{scc}|^3)$, esse limite cúbico impede que a estrutura sofra a degradação de pior caso na ordem de $O(|V|^4)$ observada na abordagem de listas, beneficiando-se do acesso imediato em tempo constante na memória.

6 Conclusão

Desta forma, conclui-se que, por meio de uma análise comparativa entre as estratégias computacionais desenvolvidas para o problema de redução transitiva e preservação de alcançabilidade, foram obtidos com êxito os resultados propostos. O estudo contrapôs a abordagem fundamentada em Lista de Adjacência, proposta originalmente pelos pesquisadores Aho, Garey e Ullman Aho *et al.* [1972], com a solução alternativa desenvolvida utilizando Matriz de Adjacência.

A análise dos dados deixou claro que não há uma estrutura de dados universalmente superior, mas sim representações adequadas a cenários específicos de densidade de grafos. Além disso, a investigação sobre o tema reforçou que o direcionamento das arestas é um requisito que fundamenta a própria definição do problema; a simetria natural desse cenário elimina a causalidade dos caminhos, gerando ambiguidade nas soluções de redução transitiva pelos algoritmos e descaracterizando a contração de ciclos, o que idealmente redireciona o foco do problema para a classe de problemas abordados pela Spanning Tree (Árvore Geradora).

Em última análise, confirma-se de forma assertiva que o algoritmo desenvolvido com Matriz de Adjacência não resolve a redução transitiva para grafos não-direcionados. A imposição dessa estrutura violaria a restrição de preservação da alcançabilidade dos vértices, gerando soluções ambíguas que invalidam a lógica de caminhos alternativos e a contração de componentes.

6.1 Delimitação de Escopo e a Fronteira NP-Difícil (MEG)

É imperativo destacar que a abordagem adotada neste trabalho soluciona a Redução Transitiva clássica em tempo polinomial, beneficiando-se da propriedade de que, na definição original de Aho *et al.* (1972), o grafo reduzido não precisa ser estritamente um subgrafo do original no caso de componentes fortemente conexos. O algoritmo tem a liberdade de formar ciclos simples ideais para restaurar a alcançabilidade interna desses componentes.

No entanto, caso a restrição do problema fosse intensificada exigindo que o grafo resultante fosse um subgrafo estrito (onde nenhuma aresta nova pode ser adicionada para otimizar os ciclos) e que possuísse o número mínimo absoluto de arestas para preservar a conectividade, o problema mudaria de classe. Essa formulação descreve o problema do Grafo Equivalente Mínimo (*Minimum Equivalent Graph* - MEG), o qual é comprovadamente **NP-Difícil** para grafos direcionados arbitrários. Portanto, a contração de ciclos via algoritmo de Tarjan não é apenas uma otimização, mas a manobra teórica que mantém a resolução do problema tratável dentro da classe P.

Declarations

Authors' Contributions

Davi Almeida, Kaiky Reis e Luís Braga contribuíram para a conceituação e a metodologia dos algoritmos de redução transitiva. Kaiky Reis, Samuel Faria e Victor Grataroli projetaram e desenvolveram a implementação sequencial e os scripts de teste de desempenho. Suzane Lima e Priscila Carvalho realizaram a análise de dados e os experimentos de topologia de grafos. Davi Almeida, Kaiky Reis e Luís Braga foram os principais redatores deste manuscrito. Todos os autores participaram ativamente da discussão dos resultados, bem como leram e aprovaram o manuscrito final.

Availability of data and materials

O código-fonte da implementação em C++ descrita neste trabalho, bem como o conjunto completo de imagens utilizadas nos testes comparativos estão disponíveis publicamente no repositório GitHub: https://github.com/GLKaiky/Transitive_reduction.

References

- Aho, A. V., Garey, M. R., and Ullman, J. D. (1972). The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137.
- Jerônimo, C., Patrocínio Jr, Z. K., Malinowski, S., Gravier, G., and Guimarães, S. J. F. (2025). Decreasing graph complexity with transitive reduction to improve temporal graph classification. *International Journal of Data Science and Analytics*, 19(2):229–242.
- Sidford, A. (2018). Lecture 2: Introduction to graph theory. Stanford University, CME 305/MS&E 316: Discrete Mathematics and Algorithms. Accessed: 27 Jun. 2026.